Hw9

$$\mathcal{E} = \left\{ x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} : x^T Q x \le 1 \right\}$$

1a

$$Q = \begin{bmatrix} 1/9 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\Rightarrow x^T Q x \Rightarrow \boxed{\frac{x_1^2}{9} + x^2 \le 9.}$$

1b

$$Ay = x \quad \text{and} \quad A = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$$

$$x^T Q x \Rightarrow (Ay)^T Q \cdot (Ay)$$

$$\Rightarrow y^T (A^T Q A) y$$

$$\Rightarrow y^T \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1/9 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} y$$

$$\Rightarrow y^T \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} y.$$

$$y = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} \Rightarrow \boxed{y_1^2 + y_2^2 \le 1}$$

1c

$$Q = \begin{bmatrix} 1 & 0.9 \\ 0.9 & 1 \end{bmatrix}$$

$$x^T Q x \le 1 \Rightarrow [x_1 \ x_2]^T \begin{bmatrix} 1 & 0.9 \\ 0.9 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$\boxed{\Rightarrow x_1^2 + 1.8 x_1 x_2 + x_2^2 \le 1.}$$

1d. $\boxed{\text{Reference}}$ lecture 12 (Newton Method)

Newton Method :-

Best $A \Rightarrow A^T \varphi A = I$

$\Rightarrow A = \varphi^{-1/2}$

ie for new $z^T z \leq 1$

1d

$$\boxed{\text{The transformation } z = Ay \text{ must satisfy } z(\varphi) z^T \leq 1}$$

This implies $A = \varphi^{-1/2}$

$$\varphi = \begin{bmatrix} 1 & 0.9 \\ 0.9 & 1 \end{bmatrix}$$

$(\varphi^{-1/2})^2 = \varphi^{-1}$

$\boxed{\text{np.linalg.eig}(\varphi)}$

Eigenvectors : $v_1 = [0.7071, 0.7071]$

$v_2 = [0.7071, -0.7071]$

Eigenvalue : $\lambda_1 = 1.9$, $\lambda_2 = 0.1$

$$\boxed{\varphi^{-1} = V \lambda^{-1} V^T \Rightarrow \varphi^{-1/2} = V \lambda^{-1/2} V^T}$$

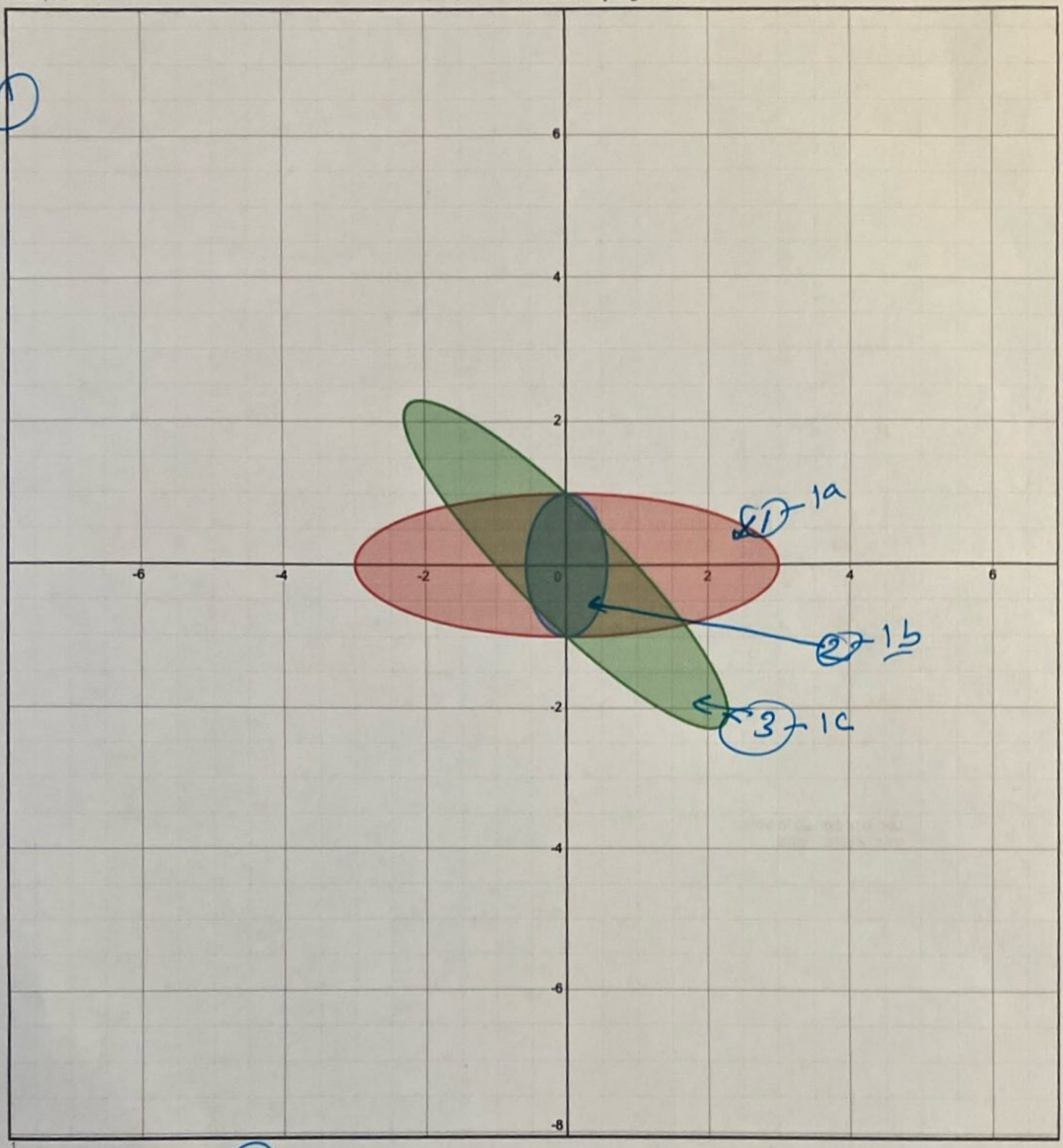
1

1d

$$\Phi^{-1} = \begin{array}{c} \begin{array}{cc} v_1 & v_2 \end{array} \\ \begin{bmatrix} 0.707 & 0.707 \\ 0.707 & -0.707 \end{bmatrix} \end{array} \begin{bmatrix} \frac{1}{\sqrt{1.9}} & 0 \\ 0 & \frac{1}{\sqrt{0.1}} \end{bmatrix} \begin{bmatrix} 0.707 & 0.707 \\ 0.707 & -0.707 \end{bmatrix}$$

$$= \begin{bmatrix} 1.94 & -1.21 \\ -1.21 & 1.94 \end{bmatrix}$$

$$\boxed{A = \Phi^{-1} = \begin{bmatrix} 1.94 & -1.21 \\ -1.21 & 1.94 \end{bmatrix}}$$



1 $\frac{x^2}{9} + y^2 \le 1$ 1 a — 1a

2 $3x^2 + y^2 \le 1$ 2 — 1b

3 $x^2 + 1.8\,xy + y^2 \le 1$ 3 — 1c

(1)

(2) $g(y) = f(Ay)$

$Ay = x$,

29/ starting at x_0, descent on $f(x)$

$Ay_0 = x_0$

Reference lecture 12 - GD is NOT Affine Invariant
solution to address this is Newton method

lets take $f(x) = x_1^2 + 9x_2^2$ from 1(a)

$$\nabla f(x) = \begin{bmatrix} 2x_1 \\ 18x_2 \end{bmatrix}, \quad \nabla^2 f(x) = \begin{bmatrix} 2 & 0 \\ 0 & 18 \end{bmatrix}$$

at $x_0 \in (1, 1) \Rightarrow \nabla f(x_0) = \begin{pmatrix} 2 \\ 18 \end{pmatrix}$

$\eta = 0.1$

$x_{k+1} = x_k - \eta \nabla f(x_k)$

22

$$x_1 = x_0 - \eta \nabla f(x_0)$$
$$= \begin{pmatrix} 1 \\ 1 \end{pmatrix} - 0.1 \begin{pmatrix} 2 \\ 18 \end{pmatrix} = \begin{pmatrix} 0.8 \\ -0.8 \end{pmatrix}$$

2

$g(y)$ is $\Rightarrow$ $g(y) = f(Ay)$.

$A = \begin{bmatrix} 3 & 0 \\ 1 & 1 \end{bmatrix}$ from 1(b)

$Ay = \begin{bmatrix} 3 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 3y_1 \\ y_1 + y_2 \end{bmatrix}$

$g(y) = f(Ay) =$

$= (3y_1)^2 + 9(y_1 + y_2)^2$

$$\boxed{g(y) = f(Ay) = 18y_1^2 + 18y_1y_2 + 9y_2^2}$$

$$\nabla g(y) = \begin{bmatrix} 36y_1 + 18y_2 \\ 18y_1 + 18y_2 \end{bmatrix}$$

$$\nabla^2 g(y) = \begin{bmatrix} 36 & 18 \\ 18 & 18 \end{bmatrix}$$

2a

$A = \begin{bmatrix} 3 & 0 \\ 1 & 1 \end{bmatrix} \Rightarrow A^{-1} = \begin{bmatrix} 1/3 & 0 \\ -1/3 & 1 \end{bmatrix}$

$x_0 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \Rightarrow y_0 = A^{-1}(x) = \begin{pmatrix} +1/3 & 0 \\ -1/3 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1/3 \\ 2/3 \end{pmatrix}$

(2)

2a From gradient descent of

$g(y) = f(Ay)$

2) $y_{k+1} = y_k - \eta \nabla g(y_k)$

$$= \begin{pmatrix} 1/3 \\ 2/3 \end{pmatrix} - 0.1 \begin{pmatrix} \overset{12}{\cancel{36}} \cdot (1/3) + \overset{6}{\cancel{18}} (2/3) \\ \cancel{18}(1/3) + \overset{6}{\cancel{18}} (2/3) \end{pmatrix}$$

$$= 0.1 \begin{pmatrix} 12 + 12 \\ 6 + 12 \end{pmatrix}$$

$$= \begin{pmatrix} 0.33 \\ 0.66 \end{pmatrix} - \begin{pmatrix} 2.4 \\ 1.8 \end{pmatrix}$$

$$= \begin{pmatrix} -2.07 \\ -1.14 \end{pmatrix}$$

$$\boxed{\cancel{x_1 = \begin{pmatrix} +0.8 \\ -0.8 \end{pmatrix} \neq \begin{pmatrix} -2.07 \\ -1.14 \end{pmatrix} = y}}$$

$$Ay = \begin{pmatrix} 3 & 0 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} -2.07 \\ -1.14 \end{pmatrix} = \begin{pmatrix} -6.21 \\ -3.21 \end{pmatrix}$$

$$\boxed{x_1 = \begin{pmatrix} 0.8 \\ -0.8 \end{pmatrix} \neq \begin{pmatrix} -6.21 \\ 3.21 \end{pmatrix} = y_1}$$

2b New Ton Method :- (undamped)

$$x_{k+1} = x_k - \left[\nabla^2 f(x)\right]^{-1} \nabla f(x)$$

$f(x)$ desmos

$$= \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \begin{bmatrix} 1/2 & 0 \\ 0 & 1/18 \end{bmatrix} \begin{bmatrix} 2\cdot(1) \\ 18\,(1) \end{bmatrix}$$

$$\boxed{x_{k+1} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}}$$

$$\nabla^2 f(x) = \begin{bmatrix} 2 & 0 \\ 0 & 18 \end{bmatrix} \Rightarrow \left[\nabla^2 f(x)\right]^{-1} = \begin{bmatrix} 1/2 & 0 \\ 0 & 1/18 \end{bmatrix}$$

$g(y)$ desmos

$$y_{k+1} = y_k - \left[\nabla^2 g(y)\right]^{-1} \nabla g(y)$$

$$\nabla^2 g(y) = \begin{bmatrix} 36 & 18 \\ 18 & 18 \end{bmatrix} \qquad y_0 = A^{-1}a = \begin{pmatrix} 1/3 \\ 2/3 \end{pmatrix}$$

from 2(a)

$$\left[\nabla^2 g(y)\right]^{-1} = \begin{bmatrix} 1/18 & -1/18 \\ -1/18 & 1/9 \end{bmatrix}$$

$$y_{k+1} = \begin{pmatrix} 1/3 \\ 2/3 \end{pmatrix} - \begin{bmatrix} 1/18 & -1/18 \\ -1/18 & 1/9 \end{bmatrix} \begin{bmatrix} \frac{\cancel{36}^{12}}{\cancel{3}} + \frac{\cancel{18}^{6}\times\cancel{2}}{\cancel{3}} \\ \frac{\cancel{18}^{6}}{\cancel{3}} + \frac{18\times 2}{3} \end{bmatrix}$$

$$= \begin{bmatrix} 1/3 \\ 2/3 \end{bmatrix} - \begin{bmatrix} 1/18 & -1/18 \\ -1/18 & 1/9 \end{bmatrix} \begin{bmatrix} 24 \\ 18 \end{bmatrix}$$

$$y_{k+1} = \begin{bmatrix} 1/3 \\ 2/3 \end{bmatrix} - \begin{bmatrix} 1/3 \\ 2/3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

(2)

$$x_0 = Ay_0 = \begin{bmatrix} 3 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Hence we can see that in Newton method (undamped)

$$\boxed{x_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} = Ay_1}$$

Refer lecture 11

(3) $\min_x \frac{1}{2}\|Ax - y\|_2^2 + \lambda\|x\|_1$

This is of the form

$$\min_x \; g(x) + h(x)$$

39

$g(x)$ is <u>smooth</u>.

$h(x)$ is <u>not smooth</u> $\Rightarrow$ and not differentiable at some points

$\min_x f(x)$ when $f(x) = g(x) + h(x)$

$$\partial f(x) = \underbrace{\nabla g(x)}_{\text{smooth.}} + \underbrace{\partial h(x)}_{\text{not smooth}}$$

$$\nabla g(x) = A^T(Ax - y)$$

$$\partial h(x) = \begin{cases} \text{sign}(x) & x \neq 0. \\ [-1, 1] & \text{if } x = 0 \end{cases}$$

Let $\partial h(x) = z$

$\Rightarrow$ At minimum there is a gradient $\underline{0}$ such that $\nabla g(x) + \partial h(x) = 0$

$f(x^*) = \min_x f(x) \iff 0 \in \partial f(x^*)$

At min the subgradient

$$\nabla g(x) + \partial h(x) = 0$$

$$\Rightarrow A^T(Ax - y) + \lambda z = 0$$

where $z = \partial h(x)$

$$\boxed{\Rightarrow -A^T(y - Ax) + \lambda z = 0}$$

3b

As already stated, to satisfy this

$$\boxed{z = \partial h(x), \; f(x^*) = \min_x f(x) \iff 0 \in \partial f(x^*)}$$

$z_i = \text{Sign}(x_i)$ for $x_i \neq 0$

$z_i = [-1, 1]$ for $x_i = 0$

(4)

a Subgradient Method :-

$$\boxed{\begin{aligned} x_{t+1} &= x_t - \eta_t g_t \\ g_t &\in \partial f(x_t) \end{aligned}}$$

*g: may not be descent direction

$\Rightarrow$ we need to keep track of the best iterate

$$f(x^t_{best}) = \min_{s \le t} f(x_s)$$

Ultimately $x_{t_{best}}$ goes to optimum.

$$\boxed{x_{t+1} = x_t - \eta_t \left(\underline{-A^T(y - Ax_t)} + \lambda z \right)}$$

z: Sign (x_t)

$$z = \begin{cases} 1 & x_t > 0 \\ 0 & x_t = 0 \\ -1 & x_t < 0 \end{cases}$$

η_t: as decreasing step size ideal

η_t: Decreasing step size Preferred

$$\boxed{x_{t+1} = x_t - \eta_t \left(\underline{A^T(Ax_t - y)} + \lambda z \right)}$$

4b $\min f(x) = g(x) + h(x)$

$\text{Prox} : \mathbb{R}^n \to \mathbb{R}^n$ for function $\underline{h}$ of x

<u>Prox operator</u>: $\text{Prox}_{\eta h}(x) \triangleq \underset{u}{\arg\min}\ h(u) + \frac{1}{2\eta}\|x-u\|_2^2$

Prox Grad :-

$$x_{t+1} = \text{Prox}_{\eta h}\left(x_t - \eta \nabla g(x_t)\right)$$

for $h(x) = \|x\|_1 = \sum |x_i|$

$\text{Prox}_{\eta h}(x) =$ Soft - Thresholding operator

$$= z \quad z_i = \begin{cases} x_i - \eta & x_i > \eta \\ 0 & -\eta \le x_i \le \eta \\ x_i + \eta & x_i < -\eta \end{cases}$$

$$\cancel{x_{t+1} = \text{Prox}_{\eta h}\left(x_t - \eta\left(-A^T(y - Ax_t)\right)\right)}$$

$$\cancel{\text{Prox}_{\eta h}(x_{\text{prox}t}) = z \quad z_t = \begin{cases} x_{\text{prox}t} - \eta & x_{\text{prox}t} > \eta \\ 0 & -\eta \le x_{\text{prox}t} \le \eta \\ x_{\text{prox}t} + \eta & \end{cases}}$$

Hw9

$$f(x) = \min_x \frac{1}{2}\|Ax-y\|_2^2 + \lambda\|x\|_1$$

4b $\min f(x) = g(x) + h(x)$

Prox $R^n \to R^n$ for function h at x

Prox operator : $\text{Prox}_{\eta h}(x) \triangleq \arg\min_u h(u) + \frac{1}{2\eta}\|x-u\|_2^2$

Prox gradient Descent.

$$x_{t+1} = \text{Prox}_{\eta h}(x_t - \eta \nabla g(x_t))$$

For $h(x) = \|x\|_1 = \sum |x_i|$

$\text{Prox}_{\eta h}(x) =$ Soft-Thresholding

$$= z \quad : z_i \begin{cases} (x_i - \eta) \text{ if } x_i > \eta \\ 0 \text{ if } -\eta \le x_i \le \eta \\ (x_i + \eta) \text{ if } x_i < \eta \end{cases}$$

$$x_{t+1} = \text{Prox}_{\eta h}\left(x_t - \eta(-A^T(y - Ax_t))\right)$$

apply Prox Operator

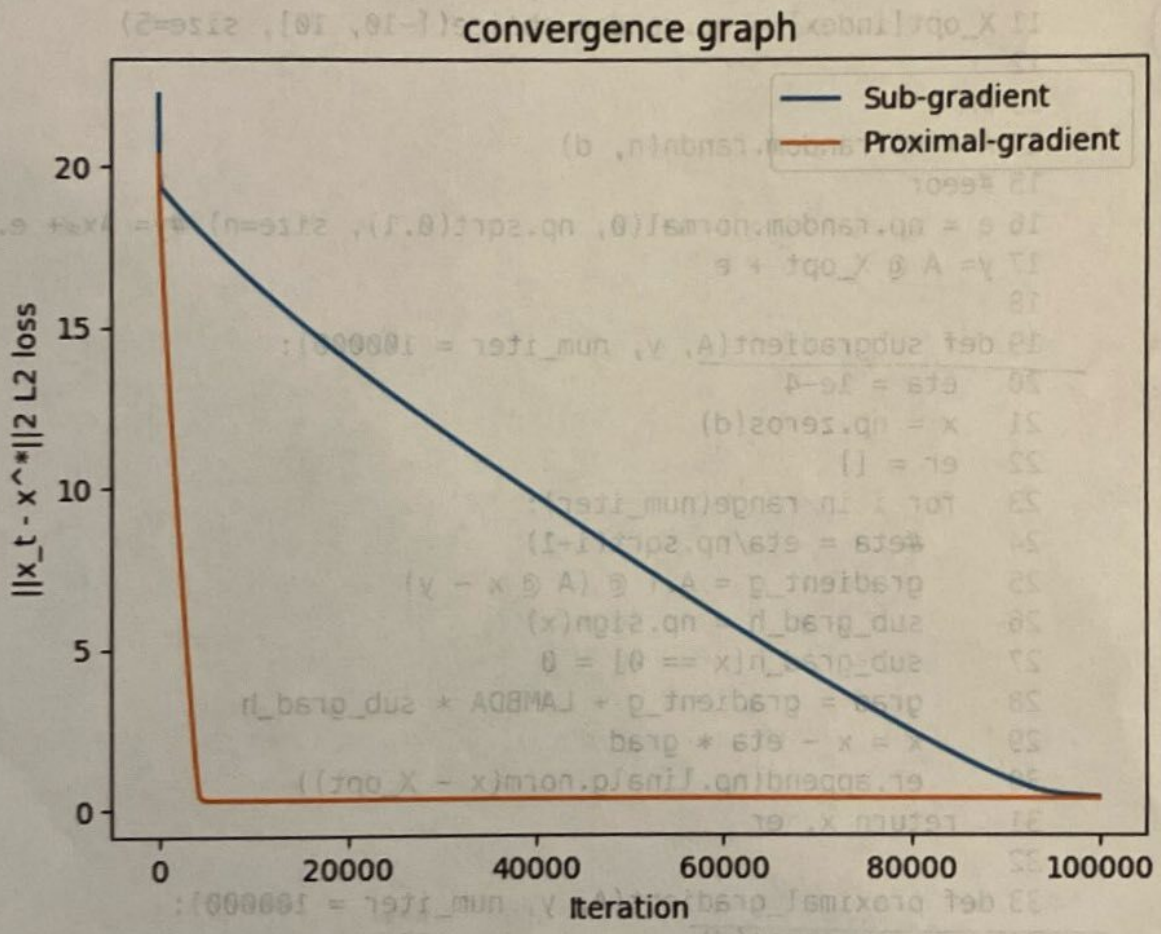
$$z \begin{cases} x_t - \eta & \text{if } x_t > \eta \\ 0 & \text{if } -\eta \le x_t \le \eta \\ x_t + \eta & \text{if } x_t < \eta \end{cases}$$

5

```python
import numpy as np
import matplotlib.pyplot as plt

n,d = 50,200
non_zero =5 #sparcity
LAMBDA = 1.0

#pick 5 indices and initialize to 10 or -10
X_opt = np.zeros(d)
index = np.random.choice(d, 5, replace=False)
X_opt[index] = np.random.choice([-10, 10], size=5)

#A
A = np.random.randn(n, d)
#eeor
e = np.random.normal(0, np.sqrt(0.1), size=n) #y= Ax*+ e.
y= A @ X_opt + e

def subgradient(A, y, num_iter = 100000):
  eta = 1e-4
  x = np.zeros(d)
  er = []
  for i in range(num_iter):
    #eta = eta/np.sqrt(i+1)
    gradient_g = A.T @ (A @ x - y)
    sub_grad_h = np.sign(x)
    sub_grad_h[x == 0] = 0
    grad = gradient_g + LAMBDA * sub_grad_h
    x = x - eta * grad
    er.append(np.linalg.norm(x - X_opt))
  return x, er

def proximal_gradient(A, y, num_iter = 100000):
  x = np.zeros(d)
  #substituting grad in grad f(y) - grad f(x) <= M * (x -y)
  M = np.linalg.norm(A, 2)**2
  eta = 1/M
  er = []
  for i  in range(num_iter):
    gradient_g = A.T @ (A @ x - y)
    z = x - eta * gradient_g
    #Line prompted by tab in colab. - soft thresholding
    #x = np.sign(z) * np.max(np.abs(z) - eta * LAMBDA, 0)#wrong we
    x = np.sign(z) * np.maximum(np.abs(z) - eta * LAMBDA, 0)
    er.append(np.linalg.norm(x - X_opt))
  return x, er
```

```
_, err_subgrad = subgradient(A, y)
_, err_prox = proximal_gradient(A, y)

plt.plot(err_subgrad, label='Sub-gradient')
plt.plot(err_prox, label='Proximal-gradient')
plt.xlabel('Iteration')
plt.ylabel(r'||x_t - x^*||2 L2 loss')
plt.legend()
plt.title('convergence graph')
plt.show()
```

5



Start coding or generate with AI.